

 	MODELO DE PROGRAMACIÓN ANUAL		MD8101
	VERSIÓN 0		Pág. 1 de 51

Programación anual del módulo profesional

Programación

Ricardo Pérez López

Curso 2026/2027

IES «Doñana», Sanlúcar de Bda.

Departamento de Informática y Comunicaciones

Jefe de Departamento: Ricardo Pérez López

Índice

1. Contextualización	4
1.1. Información general	4
1.2. Marco legislativo	4
1.3. Perfil profesional	4
1.4. Análisis de las pruebas iniciales y adecuación de los contenidos por grupos . . .	4
2. Objetivos generales	5
3. Resultados de aprendizaje y criterios de evaluación	5
4. Contenidos y temporalización	10
4.1. Cuadro resumen	11
4.2. Esquema detalle	11
4.3. Correspondencia con resultados de aprendizaje y criterios de evaluación	30
5. Evaluación y calificación	32
5.1. Valoración general de los instrumentos de evaluación y calificación	34
5.2. Calificación	35
5.2.1. De un criterio de evaluación	35
5.2.2. De un resultado de aprendizaje	36
5.2.3. Calificaciones parciales	37
5.2.4. Calificación final	38
5.3. Ponderación de los criterios de evaluación	39
5.4. Medidas de recuperación y mejora de la calificación final	41
5.4.1. Prueba final	42
5.4.2. Mejora de la calificación final	42
6. Evaluación de la práctica docente	43
7. Orientaciones pedagógicas	43
8. Orientaciones metodológicas	44
9. Recursos	45
9.1. Hardware	45
9.2. Software	46
9.3. Online	46
9.4. Bibliografía	46
9.4.1. Principal	46

9.4.2. Complementaria	46
10. Atención a la diversidad	47
11. Temas transversales	47
12. Actuaciones para desarrollar la perspectiva de género	48
12.1. Actuaciones generales permanentes	49
13. Funciones del profesor de desdoble	50

 Junta de Andalucía	 IES DOÑANA	MODELO DE PROGRAMACIÓN ANUAL		MD8101	
				VERSIÓN 0	Pág. 4 de 51

1. Contextualización

1.1. Información general

- **Equivalencia en créditos ECTS:** 15.
- **Código:** 0485.
- **Duración total:** 256 horas (32 semanas)
- **Carga lectiva semanal:** 8 horas

1.2. Marco legislativo

- **Normativa de aplicación:** [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11].

1.3. Perfil profesional

La formación del módulo contribuye a alcanzar las **competencias profesionales, personales y sociales** de este título que se relacionan a continuación:

- e) Desarrollar aplicaciones web con acceso a bases de datos utilizando lenguajes, objetos de acceso y herramientas de mapeo adecuados a las especificaciones.
- f) Integrar contenidos en la lógica de una aplicación web, desarrollando componentes de acceso a datos adecuados a las especificaciones.
- h) Desarrollar componentes multimedia para su integración en aplicaciones web, empleando herramientas específicas y siguiendo las especificaciones establecidas.
- j) Desarrollar e integrar componentes software en el entorno del servidor web, empleando herramientas y lenguajes específicos, para cumplir las especificaciones de la aplicación.
- o) Gestionar y/o realizar el mantenimiento de los recursos de su área en función de las cargas de trabajo y el plan de mantenimiento.

1.4. Análisis de las pruebas iniciales y adecuación de los contenidos por grupos

En las primeras semanas del curso se realizará una prueba inicial diagnóstica con el fin de conocer el nivel de competencias y conocimientos previos del alumnado en relación con los contenidos del módulo. Con el análisis individual de esta prueba se conseguirá detectar diferencias de nivel entre el alumnado e identificar posibles carencias en conocimientos previos requeridos.

 Junta de Andalucía	 IES DOÑANA	MODELO DE PROGRAMACIÓN ANUAL		MD8101	
				VERSIÓN 0	Pág. 5 de 51

Con los resultados de la evaluación inicial se podrán ajustar la secuenciación y profundización de los contenidos a impartir, adaptando las primeras unidades a las necesidades reales del grupo, de manera que aquellos estudiantes que presenten mayores carencias se puedan acomodar al desarrollo del curso. En caso de detectarse dificultades significativas o ritmos de aprendizaje diferentes, se podrán aplicar, a discreción del profesor, medidas tales como:

- Realización de tutorías individualizadas o en grupo para afianzar conocimientos.
- Refuerzo inicial mediante actividades prácticas guiadas.
- Materiales adaptados con ejercicios de nivel progresivo.

2. Objetivos generales

1. La formación del módulo contribuye a alcanzar los **objetivos generales** de este ciclo formativo que se relacionan a continuación:
 - f) Seleccionar lenguajes, objetos y herramientas, interpretando las especificaciones para desarrollar aplicaciones web con acceso a bases de datos.
 - g) Utilizar lenguajes, objetos y herramientas, interpretando las especificaciones para desarrollar aplicaciones web con acceso a bases de datos.
 - h) Generar componentes de acceso a datos, cumpliendo las especificaciones, para integrar contenidos en la lógica de una aplicación web.
 - j) Emplear herramientas y lenguajes específicos, siguiendo las especificaciones, para desarrollar componentes multimedia.
 - l) Utilizar herramientas y lenguajes específicos, cumpliendo las especificaciones, para desarrollar e integrar componentes software en el entorno del servidor web.
 - q) Programar y realizar actividades para gestionar el mantenimiento de los recursos informáticos.

3. Resultados de aprendizaje y criterios de evaluación

Los resultados de aprendizaje del módulo y sus criterios de evaluación asociados son los que se describen a continuación, donde los marcados con la etiqueta *#dual* son los programados

 Junta de Andalucía	 IES DOÑANA	MODELO DE PROGRAMACIÓN ANUAL		MD8101	
				VERSIÓN 0	Pág. 6 de 51

para su impartición y evaluación en la fase de formación dual en las empresas u organismos equiparados:

[RA₁] Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

Criterios de evaluación:

- CE_{1.a}) Se han identificado los bloques que componen la estructura de un programa informático.
- CE_{1.b}) Se han utilizado entornos integrados de desarrollo.
- CE_{1.c}) Se han creado proyectos de desarrollo de aplicaciones.
- CE_{1.d}) Se han identificado los distintos tipos de variables y la utilidad específica de cada uno.
- CE_{1.e}) Se ha modificado el código de un programa para crear y utilizar variables.
- CE_{1.f}) Se han creado y utilizado constantes y literales.
- CE_{1.g}) Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje.
- CE_{1.h}) Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas.
- CE_{1.i}) Se han introducido comentarios en el código.

[RA₂] Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.

Criterios de evaluación:

- CE_{2.a}) Se han identificado los fundamentos de la programación orientada a objetos.
- CE_{2.b}) Se han instanciado objetos a partir de clases predefinidas.
- CE_{2.c}) Se han utilizado constructores.
- CE_{2.d}) Se han utilizado métodos y propiedades de los objetos.
- CE_{2.e}) Se han escrito llamadas a métodos estáticos.
- CE_{2.f}) Se han utilizado parámetros en la llamada a métodos.
- CE_{2.g}) Se han incorporado y utilizado librerías de objetos.
- CE_{2.h}) Se han escrito programas simples.
- CE_{2.i}) Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples.

 Junta de Andalucía	 IES DOÑANA	MODELO DE PROGRAMACIÓN ANUAL		MD8101	
				VERSIÓN 0	Pág. 7 de 51

[RA₃] Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.

Criterios de evaluación:

CE_{3.a}) Se ha escrito y probado código que haga uso de estructuras de selección.

CE_{3.b}) Se han utilizado estructuras de repetición.

CE_{3.c}) Se han reconocido las posibilidades de las sentencias de salto.

CE_{3.d}) Se ha escrito código utilizando control de excepciones.

CE_{3.e}) Se han creado excepciones.

CE_{3.f}) Se han creado programas ejecutables utilizando diferentes estructuras de control.

CE_{3.g}) Se han probado y depurado los programas.

CE_{3.h}) Se han utilizado aserciones para la detección y corrección de errores durante la fase de desarrollo.

CE_{3.i}) Se ha comentado y documentado el código.

[RA₄] Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.

Criterios de evaluación:

CE_{4.a}) Se ha reconocido la sintaxis, estructura y componentes típicos de una clase.

CE_{4.b}) Se han definido clases.

CE_{4.c}) Se han definido propiedades y métodos.

CE_{4.d}) Se han creado y utilizado métodos estáticos.

CE_{4.e}) Se han creado constructores.

CE_{4.f}) Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.

CE_{4.g}) Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.

CE_{4.h}) Se han definido y utilizado clases heredadas.

CE_{4.i}) Se han creado y utilizado conjuntos y librerías de clases.

[RA₅] Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases.

Criterios de evaluación:

CE_{5.a}) Se ha utilizado la consola para realizar operaciones de entrada y salida de información.

 Junta de Andalucía		MODELO DE PROGRAMACIÓN ANUAL		MD8101	
				VERSIÓN 0	Pág. 8 de 51

CE_{5.b}) Se han aplicado formatos en la visualización de la información.

CE_{5.c}) Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas.

CE_{5.d}) Se han utilizado ficheros para almacenar y recuperar información.

CE_{5.e}) Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros.

CE_{5.f}) Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples.

CE_{5.g}) Se han programado controladores de eventos.

CE_{5.h}) Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información.

[RA₆] Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos.

Criterios de evaluación:

CE_{6.a}) Se han escrito programas que utilicen matrices (*arrays*).

CE_{6.b}) Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados.

CE_{6.c}) Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles.

CE_{6.d}) Se han utilizado listas para almacenar y procesar información.

CE_{6.e}) Se han utilizado iteradores para recorrer los elementos de las listas.

CE_{6.f}) Se han utilizado operaciones agregadas para el manejo de información almacenada en colecciones.

CE_{6.g}) Se han creado clases y métodos genéricos.

CE_{6.h}) Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.

CE_{6.i}) Se han identificado las clases relacionadas con el tratamiento de documentos escritos en diferentes lenguajes de intercambio de datos.

CE_{6.j}) Se han realizado programas que realicen manipulaciones sobre documentos escritos en diferentes lenguajes de intercambio de datos.

[RA₇] Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.

Criterios de evaluación:

- CE_{7.a}) Se han identificado y evaluado los escenarios de utilización de la herencia y la composición.
- CE_{7.b}) Se han identificado los conceptos de herencia, superclase y subclase.
- CE_{7.c}) Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos.
- CE_{7.d}) Se ha reconocido la incidencia de los constructores en la herencia.
- CE_{7.e}) Se han creado clases heredadas que sobrescriban la implementación de métodos de la superclase.
- CE_{7.f}) Se han diseñado y aplicado jerarquías de clases.
- CE_{7.g}) Se han probado y depurado las jerarquías de clases.
- CE_{7.h}) Se han identificado y evaluado los escenarios de uso de interfaces.
- CE_{7.i}) Se han realizado programas que implementen y utilicen jerarquías de clases.
- CE_{7.j}) Se ha comentado y documentado el código.

[RA₈] Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información.

Criterios de evaluación:

- CE_{8.a}) Se han identificado las características de las bases de datos orientadas a objetos.
- CE_{8.b}) Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos.
- CE_{8.c}) Se han instalado sistemas gestores de bases de datos orientados a objetos.
- CE_{8.d}) Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores para la gestión de la información almacenada.
- CE_{8.e}) Se han creado bases de datos y las estructuras necesarias para el almacenamiento de objetos.
- CE_{8.f}) Se han programado aplicaciones que almacenen objetos en las bases de datos creadas.
- CE_{8.g}) Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos.

CE_{8.h}) Se han realizado programas para almacenar y gestionar tipos de datos estructurados, compuestos y relacionados.

[RA₉] Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos. #dual

Criterios de evaluación:

CE_{9.a}) Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos.

CE_{9.b}) Se han programado conexiones con bases de datos.

CE_{9.c}) Se ha escrito código para almacenar información en bases de datos.

CE_{9.d}) Se han creado programas para recuperar y mostrar información almacenada en bases de datos.

CE_{9.e}) Se han efectuado borrados y modificaciones sobre la información almacenada.

CE_{9.f}) Se han creado aplicaciones que muestren la información almacenada en bases de datos.

CE_{9.g}) Se han creado aplicaciones para gestionar la información presente en bases de datos.

4. Contenidos y temporalización

La duración de cada unidad didáctica se calcula como el periodo de tiempo transcurrido desde su fecha de inicio hasta la fecha de inicio de la unidad didáctica inmediatamente siguiente (o hasta el final del curso, en el caso de la última unidad didáctica). Dicha duración podrá cambiar en función de las circunstancias cuando el profesor lo estime conveniente —por ejemplo, para la realización de actividades, ejercicios y prácticas en clase.

Los contenidos marcados con la etiqueta #dual son contenidos programados para su impartición en la fase de formación dual en las empresas u organismos equiparados.

Los contenidos marcados con la etiqueta #opcional son contenidos complementarios que sólo se impartirán si hay tiempo suficiente para ello y nunca a costa de otros contenidos no opcionales. También podrán ser usados como elementos a desarrollar para el alumnado con altas capacidades o que manifieste un ritmo de aprendizaje superior al del resto del grupo/clase.

Todas las fechas se muestran en formato ISO 8601 (año-mes-día).

4.1. Cuadro resumen

Unidad didáctica	Inicio estimado
1. Introducción #ev1	2026-09-16
2. Lenguajes de programación #ev1	2026-09-23
3. Expresiones #ev1	2026-09-30
4. Tipos y operaciones #ev1	2026-10-07
5. Programación funcional #ev1	2026-10-14
6. Abstracciones funcionales #ev1	2026-10-21
7. El modelo de entorno #ev1	2026-10-28
8. Programación imperativa (I) #ev1	2026-11-04
9. Programación imperativa (II) #ev1	2026-11-11
10. Programación estructurada #ev1	2026-11-18
11. Programación procedimental #ev1	2026-11-25
12. Procesamiento de datos de alto nivel #ev2	2027-01-12
13. Secuencias #ev2	2027-01-19
14. Colecciones no secuenciales #ev2	2027-01-26
15. Entrada y salida por archivos #ev2	2027-02-02
16. Programación modular #ev2	2027-02-09
17. Abstracción de datos #ev2	2027-02-16
18. Programación orientada a objetos (I) #ev2	2027-02-23
19. Programación orientada a objetos (II) #ev2	2027-03-02
20. Relaciones entre clases #ev2	2027-03-09
21. Interfaces gráficas de usuario #ev3	2027-03-30
22. Bases de datos orientadas a objetos #ev3	2027-04-06
23. Bases de datos relacionales #ev3 #dual	2027-04-13

4.2. Esquema detalle

El título de cada unidad va etiquetado con los resultados de aprendizaje y los criterios de evaluación relacionados con dicha unidad. Cuando un resultado de aprendizaje no lleva asociado ningún criterio de evaluación, significa que dicho resultado de aprendizaje se trabaja en la unidad didáctica pero no es el elemento fundamental de evaluación.

1 INTRODUCCIÓN #ce1a #ce2a #ev1 #ra1 #ra2 (est: 2026-09-16)

1.1. Conceptos básicos

1.1.1. Informática

- Procesamiento automático

1.1.2. Ordenador

- Definición
- Funcionamiento básico
 - Elementos funcionales
 - Ciclo de instrucción
- Representación de información
 - Sistema binario
 - Conjuntos de caracteres
 - ASCII
 - Unicode

1.1.3. Problema

- Generalización
- Ejemplares de un problema
- Dominio de definición
- Jerarquías de generalización

1.1.4. Algoritmo

- Definición
- Características
- Cualidades deseables
- Representación
 - Ordinograma
 - Pseudocódigo
- Computabilidad
- Corrección
- Complejidad

1.1.5. Programa

1.1.6. Lenguaje de programación

1.2. Paradigmas de programación

1.2.1. Definición

1.2.2. Imperativo

- Estructurado
- Orientado a objetos

1.2.3. Declarativo

- Funcional
- Lógico
- De bases de datos

1.3. Resolución de problemas mediante programación

- 1.3.1. Especificación
- 1.3.2. Análisis del problema
- 1.3.3. Diseño del algoritmo
- 1.3.4. Verificación
- 1.3.5. Estudio de la eficiencia
- 1.3.6. Codificación
 - Implementación
- 1.3.7. Traducción y ejecución
- 1.3.8. Pruebas
- 1.3.9. Depuración
- 1.3.10. Documentación
- 1.3.11. Mantenimiento
- 1.3.12. Ingeniería del software

2 LENGUAJES DE PROGRAMACIÓN *#ce1b #ce1c #ce2a #ev1 #ra1 #ra2* (est: 2026-09-23)

2.1. Definición

- 2.1.1. Sintaxis
 - Notación EBNF
- 2.1.2. Semántica estática
- 2.1.3. Semántica dinámica

2.2. Evolución histórica *#opcional*

2.3. Clasificación

- 2.3.1. Por nivel
- 2.3.2. Por generación
- 2.3.3. Por propósito
- 2.3.4. Por paradigma

2.4. Traductores

- 2.4.1. Traductores
- 2.4.2. Compiladores
 - Ensambladores
- 2.4.3. Intérpretes
 - Interactivos (*REPL*)
- 2.4.4. Licencias de software
 - Dominio público
 - Software privativo

- Software gratuito (*freeware*)
- Software libre
 - Software libre y software de código abierto
 - Licencias de software libre y de código abierto

2.5. Entornos integrados de desarrollo

- 2.5.1. Definición
- 2.5.2. Editores de textos
- 2.5.3. Editores vs. IDE
- 2.5.4. Thonny

2.6. El lenguaje de programación Python

- 2.6.1. Historia
- 2.6.2. Características principales
- 2.6.3. Instalación
- 2.6.4. Funcionamiento del intérprete
 - Entrar y salir del intérprete interactivo

3 EXPRESIONES *#ce1d #ce1e #ce1f #ce1g #ce2b #ce2c #ce2d #ce2e #ce2f #ce2g #ce2h #ce2i #ev1 #ra1 #ra2* (est: 2026-09-30)

3.1. Elementos de un programa

- 3.1.1. Expresiones y sentencias
- 3.1.2. Sintaxis y semántica de las expresiones

3.2. Valores

- 3.2.1. Información, datos, tipos y valores
- 3.2.2. Evaluación de expresiones
- 3.2.3. Expresión canónica y forma normal
- 3.2.4. Formas normales y evaluación
- 3.2.5. Árboles sintácticos y evaluación
- 3.2.6. Literales
- 3.2.7. Identificadores

3.3. Operaciones

- 3.3.1. Clasificación
- 3.3.2. Operadores
 - Aridad de operadores
 - Notación de los operadores
 - Paréntesis
 - Prioridad de operadores
 - Asociatividad de operadores

- Paréntesis y orden de evaluación
- Tipos de operandos

3.3.3. Funciones

- Evaluación de expresiones con llamadas a funciones
- Composición de operaciones

3.3.4. Métodos

4 TIPOS Y OPERACIONES #ce1d #ce1f #ce1g #ce1h #ev1 #ra1 (est: 2026-10-07)

4.1. Sistemas de tipos

- 4.1.1. Concepto
- 4.1.2. Errores de tipos
- 4.1.3. Tipado estático vs. dinámico
- 4.1.4. Tipado fuerte vs. débil
- 4.1.5. La función `type`
- 4.1.6. Conversión de tipos
- 4.1.7. Tipos de datos básicos

4.2. Números

- 4.2.1. Los tipos `int` y `float`
- 4.2.2. Operadores aritméticos
- 4.2.3. Funciones numéricas predefinidas
- 4.2.4. Funciones matemáticas y módulos
 - El módulo `operator`

4.3. Cadenas

- 4.3.1. El tipo `str`
- 4.3.2. Operadores de cadenas
- 4.3.3. Funciones predefinidas de cadenas
- 4.3.4. Métodos predefinidos de cadenas

4.4. Funciones

- 4.4.1. El tipo *función*

4.5. Álgebra de Boole

- 4.5.1. El tipo de dato *booleano*
- 4.5.2. Operadores relacionales
- 4.5.3. Operadores lógicos
 - Tablas de verdad
- 4.5.4. Axiomas
- 4.5.5. Teoremas fundamentales

- 4.5.6. Notación algebraica
- 4.5.7. Equivalencia lógica
- 4.5.8. El operador ternario
- 4.6. Otros conceptos sobre operaciones
 - 4.6.1. Árboles sintácticos y evaluación
 - 4.6.2. Tipos polimórficos y operaciones polimórficas
 - 4.6.3. Sobrecarga de operaciones
 - 4.6.4. Equivalencia entre formas de operaciones
 - 4.6.5. Igualdad de operaciones

5 PROGRAMACIÓN FUNCIONAL *#ce1e #ce1f #celi #ev1 #ra1* (est: 2026–10–14)

- 5.1. Introducción
 - 5.1.1. Concepto
 - 5.1.2. Transparencia referencial
 - Efectos laterales
 - 5.1.3. Modelo de ejecución
 - Modelo de sustitución
- 5.2. Definiciones
 - 5.2.1. Identificadores y ligaduras (*binding*)
 - Ligaduras irrompibles
 - Inmutabilidad
 - Reglas léxicas
 - Tipo de un identificador
 - Anotaciones de tipo en definiciones
 - Otras formas de crear ligaduras
 - 5.2.2. Espacios de nombres
 - 5.2.3. Marcos (*frames*)
 - 5.2.4. Evaluación de expresiones con identificadores
 - Resolución de identificadores
- 5.3. *Scripts*
- 5.4. Documentación interna
 - 5.4.1. Concepto
 - 5.4.2. Comentarios
 - 5.4.3. Identificadores significativos

6 **ABSTRACCIONES FUNCIONALES** #ce4d #ev1 #ra4 (est: 2026–10–21)

6.1. Abstracciones lambda

6.1.1. Expresiones lambda

6.1.2. Parámetros y cuerpos

6.1.3. Aplicación funcional

- Evaluación de una aplicación funcional
- Funciones con nombre
- Composición de funciones

6.1.4. Identificadores cuantificados y libres de una expresión lambda

6.1.5. Anotaciones de tipo en funciones

6.2. Abstracciones funcionales

6.2.1. Encapsulación y cajas negras

6.2.2. Las funciones como abstracciones

- Control de la complejidad
- Definición de conceptos abstractos
- Generalización

6.2.3. Pureza

6.2.4. Especificaciones de funciones

6.3. Recursividad #opcional

6.3.1. Funciones y procesos

- Funciones *ad-hoc*

6.3.2. Funciones recursivas

- Definición
- Casos base y casos recursivos
- El factorial
- Diseño de funciones recursivas
 - Identificación de casos base
 - Descomposición (reducción) del problema
 - Pensamiento optimista
- Recursividad lineal
 - Procesos recursivos lineales #opcional
 - Procesos iterativos lineales #opcional
- Recursividad múltiple
- Recursividad final y no final #opcional

6.3.3. Tipos de datos recursivos

- Concepto

- Cadenas
- Tuplas
 - Conversión a tupla

7 EL MODELO DE ENTORNO *#ce1d #ce1e #ce1f #ce1g #ev1 #ra1* (est: 2026-10-28)

7.1. Ámbitos

7.1.1. Ámbitos léxicos

- Ámbito global

7.1.2. Ámbito de una instrucción y de una ligadura

- Visibilidad
- Tiempo de vida
- Almacenamiento

7.1.3. Ámbito de un identificador

7.1.4. Ámbito de un parámetro

7.1.5. Ámbito de un identificador cuantificado

7.1.6. Ámbito de un identificador libre

7.2. Entorno

7.2.1. Definición

7.2.2. Ámbitos, marcos y entornos

7.3. Evaluación con entornos

7.3.1. Evaluación de expresiones con entornos

7.3.2. Evaluación de expresiones lambda con entornos

- Ligaduras *sombreadas*
- Renombrado de parámetros
- Visualización en *Pythontutor*

7.3.3. Resolución de atributos de objetos

7.4. La pila de control

7.5. Estrategias de evaluación *#opcional*

7.5.1. Estrategias de evaluación

7.5.2. Orden de evaluación

- Orden aplicativo
- Orden normal

7.5.3. Evaluación estricta y no estricta

8 PROGRAMACIÓN IMPERATIVA (I) *#ce1e #ev1 #ra1* (est: 2026-11-04)

8.1. Modelo de ejecución

8.1.1. Máquina de estados

- 8.1.2. Sentencias
- 8.1.3. Secuencia de sentencias
- 8.2. Asignación destructiva
 - 8.2.1. Valores y referencias
 - 8.2.2. Variables
 - 8.2.3. Estado
 - 8.2.4. Marcos en programación imperativa
 - 8.2.5. Sentencia de asignación
 - 8.2.6. La sentencia del
 - 8.2.7. Alias de variables y valores idénticos
 - 8.2.8. Recolección de basura
 - 8.2.9. Evaluación de expresiones con variables
 - 8.2.10. Tipado estático vs. dinámico
 - 8.2.11. Asignación compuesta
 - 8.2.12. Asignación múltiple
 - 8.2.13. Constantes
- 8.3. Saltos
 - 8.3.1. Incondicionales
 - 8.3.2. Condicionales

9 **PROGRAMACIÓN IMPERATIVA (II)** #ce5a #ce5c #ev1 #ra5 (est: 2026-11-11)

- 9.1. Mutabilidad
 - 9.1.1. Estado de un dato
 - 9.1.2. Tipos mutables e inmutables
 - Valores inmutables
 - Secuencias
 - Valores mutables: listas
 - 9.1.3. Alias de variables y valores mutables
 - 9.1.4. Identidad
 - is
 - Identidad e igualdad
- 9.2. Cambios de estado ocultos
 - 9.2.1. Funciones puras
 - 9.2.2. Funciones impuras
 - 9.2.3. Efectos laterales
 - 9.2.4. Ejecución de *scripts* por lotes
 - Argumentos de la línea de órdenes

9.3. Entrada y salida

9.3.1. Conceptos básicos

9.3.2. Entrada y salida por consola

- print
 - Paso de argumentos por palabras clave
 - Argumentos por defecto
 - El valor None
- input

10 PROGRAMACIÓN ESTRUCTURADA (est: 2026-11-18)

10.1. Aspectos teóricos de la programación estructurada

10.1.1. Programación estructurada

10.1.2. Programa restringido

10.1.3. Programa propio

10.1.4. Estructuras de control

10.1.5. Programa estructurado

- Ventajas de los programas estructurados

10.1.6. Teorema de Böhm-Jacopini

10.2. Estructuras básicas de control en Python

10.2.1. Secuencia

10.2.2. Selección

10.2.3. Iteración

10.2.4. Otras sentencias de control

- break
- continue
- match
- Excepciones
 - Gestión de excepciones
 - Lanzamiento de excepciones

10.3. Metodología de la programación estructurada

10.3.1. Recursos abstractos

10.3.2. Diseño descendente

10.3.3. Refinamiento sucesivo

11 PROGRAMACIÓN PROCEDIMENTAL *#ce3a #ce3b #ce3c #ce3d #ce3e #ce3f #ce3g #ce3h #ce3i #ev1 #ra3* (est: 2026-11-25)

11.1. Conceptos básicos

11.1.1. Subprogramas

11.1.2. El paradigma de programación procedimental

11.1.3. Procedimientos y refinamiento sucesivo

11.2. Funciones imperativas

11.2.1. Introducción

11.2.2. Definición de funciones imperativas

11.2.3. Llamadas a funciones imperativas

11.2.4. Paso de argumentos

- Argumentos por defecto
- Paso de argumentos por palabra clave

11.2.5. La sentencia return

11.3. Ámbitos en funciones funciones

11.3.1. Ámbito de variables

- Variables locales
- Variables globales
 - global
 - Efectos laterales

11.3.2. Funciones locales a funciones

- nonlocal

11.4. Calidad *#opcional*

11.4.1. Documentación

- *Docstrings*
- pydoc

11.4.2. Pruebas

- doctest
- pytest

11.4.3. Análisis estático

12 PROCESAMIENTO DE DATOS DE ALTO NIVEL *#ce6b #ce6c #ce6e #ce6f #ce6g #ev2 #ra6* (est: 2027-01-12)

12.1. Colecciones

12.1.1. Composición

12.1.2. Conceptos básicos

12.1.3. Clasificación

12.1.4. *Hashables*

12.2. Flujos perezosos

12.2.1. Iterables e iteradores

12.2.2. El bucle for

12.2.3. Iteradores sobre iterables mutables

12.2.4. range

12.2.5. Funciones que devuelven iteradores

- El módulo `itertools`
- `enumerate`
- `zip`
- `reversed`

12.3. Funciones de orden superior

12.3.1. Concepto

12.3.2. map

12.3.3. filter

12.3.4. reduce

12.3.5. Procesamiento de flujos

12.3.6. Expresiones generadoras

12.4. Funciones genéricas

12.4.1. Definición y uso

13 SECUENCIAS #ce5b #ce6a #ce6b #ce6c #ce6d #ce6h #ev2 #ra5 #ra6 (est: 2027-01-19)

13.1. Concepto de secuencia

13.1.1. Definición

13.1.2. Operaciones comunes

13.2. Inmutables

13.2.1. Cadenas (`str`)

- Formateado de cadenas
- Expresiones regulares
 - Clases de caracteres
 - Secuencias de barra invertida
 - Repeticiones
 - Opcionalidad y agrupamiento
 - Métodos del módulo `re`

13.2.2. Tuplas

13.2.3. Rangos

13.3. Mutables

13.3.1. Listas

- Listas por compresión

13.3.2. Operaciones mutadoras

14 COLECCIONES NO SECUENCIALES #ce6b #ce6c #ce6i #ce6j #ev2 #ra6 (est: 2027-01-26)

14.1. Conjuntos (set y frozenset)

14.1.1. Definición

14.1.2. Conjuntos por comprensión

14.1.3. Operaciones

14.1.4. Operaciones sobre conjuntos mutables

14.1.5. Recorrido de conjuntos

14.2. Diccionarios (dict)

14.2.1. Definición

14.2.2. Diccionarios por comprensión

14.2.3. Operaciones

14.2.4. Recorrido de diccionarios

14.3. Documentos XML

14.3.1. Definición

14.3.2. Acceso

14.3.3. Modificación

15 ENTRADA Y SALIDA POR ARCHIVOS #ce5c #ce5d #ce5e #ev2 #ra5 (est: 2027-02-02)

15.1. Introducción

15.2. Operaciones de apertura y cierre

15.2.1. open

15.2.2. close

15.2.3. Gestores de contexto

15.3. Operaciones de lectura

15.3.1. read

15.3.2. readline

15.3.3. readlines

15.3.4. Archivos como iterables

15.4. Operaciones de escritura

15.4.1. write

15.4.2. writelines

15.5. Otras operaciones

15.5.1. seek y tell

16 PROGRAMACIÓN MODULAR #ce4i #ev2 #ra4 (est: 2027-02-09)

16.1. Introducción

16.1.1. Modularidad

16.1.2. Descomposición de problemas

16.1.3. Beneficios de la modularidad

16.2. Diseño modular

16.2.1. Creadores y usuarios

16.2.2. Partes de un módulo

- Interfaz
 - Especificación
- Implementación

16.2.3. Diagramas de estructura

16.3. Programación modular en Python

16.3.1. *Scripts* como módulos

- *Docstrings*

16.3.2. Importación de módulos

16.3.3. Módulos como *scripts*

16.3.4. La librería estándar

16.4. Criterios de descomposición modular

16.4.1. Abstracción

16.4.2. Ocultación de información

16.4.3. Independencia funcional

- Cohesión
- Acoplamiento

16.4.4. Reusabilidad

- Estrategia sándwich

16.4.5. Principios *SOLID*

17 ABSTRACCIÓN DE DATOS #ce4b #ce4e #ev2 #ra4 (est: 2027-02-16)

17.1. Introducción

17.1.1. Introducción

17.1.2. Tipos abstractos de datos

17.2. Especificaciones

17.2.1. Sintaxis

17.2.2. Operaciones

17.2.3. Ejemplos

17.3. Implementaciones

17.3.1. Diseño ascendente

17.3.2. Implementaciones

17.4. Niveles y barreras de abstracción

17.4.1. Niveles de abstracción

17.4.2. Barreras de abstracción

17.4.3. Propiedades de los datos

17.5. Las funciones como datos

17.5.1. Clausuras

17.5.2. Representación funcional

17.5.3. Estado interno

17.5.4. Paso de mensajes

17.5.5. Especificación de datos abstractos con estado interno

17.6. Abstracción de datos y modularidad

17.6.1. El tipo abstracto como módulo

18 PROGRAMACIÓN ORIENTADA A OBJETOS (I) *#ce3h #ce4a #ce4b #ce4c #ce4d #ce4e #ce4f #ce4g #ce4i #ce6g #ev2 #ra3 #ra4 #ra6* (est: 2027-02-23)

18.1. Introducción

18.1.1. Recapitulación

18.1.2. Objetos

18.2. Conceptos básicos

18.2.1. Atributos

18.2.2. Clases

- Instancias

18.2.3. Estado

- Variables de instancia

18.2.4. La antisimetría dato-objeto

18.3. Paso de mensajes

18.3.1. Resolución de atributos

18.3.2. Ejecución de métodos

18.3.3. Definición de métodos

- Entorno durante la ejecución de métodos

18.3.4. Métodos *mágicos* y constructores

19 PROGRAMACIÓN ORIENTADA A OBJETOS (II) *#ce3h #ce4a #ce4b #ce4c #ce4d #ce4e #ce4f #ce4g #ce4i #ce6g #ev2 #ra3 #ra4 #ra6* (est: 2027-03-02)

19.1. Identidad e igualdad

19.1.1. Identidad

19.1.2. Igualdad

- `__eq__`
- `__hash__`

19.1.3. Otros métodos mágicos

- `__repr__`
- `__str__`
- `__iter__` y `__next__`

19.2. Encapsulación

19.2.1. La encapsulación como mecanismo de agrupamiento

19.2.2. La encapsulación como mecanismo de protección de datos

- Visibilidad
- Accesores y mutadores
- Propiedades
- Invariantes de clase
- Interfaz y especificación de una clase
- Asertos

19.3. Miembros de clase

19.3.1. Variables de clase

19.3.2. Métodos estáticos

19.4. Clases genéricas y métodos genéricos

19.4.1. Definición y uso

20 RELACIONES ENTRE CLASES *#ce4a #ce4b #ce4c #ce4d #ce4e #ce4f #ce4g #ce4h #ce4i #ce7a #ce7b #ce7c #ce7d #ce7e #ce7f #ce7g #ce7h #ce7i #ce7j #ev2 #ra4 #ra7* (est: 2027-03-09)

20.1. Relaciones básicas

20.1.1. Introducción

20.1.2. Asociación

20.1.3. Dependencia

20.1.4. Agregación

20.1.5. Composición

20.2. Herencia

20.2.1. Generalización

20.2.2. Modos

- Herencia simple
- Visibilidad de miembros y herencia
 - Visibilidad protegida
- La clase object
- Métodos de clase
- Herencia múltiple

20.3. Polimorfismo

20.3.1. Concepto

20.3.2. Principio de sustitución de Liskov

20.3.3. *Duck typing*

20.3.4. Sobreescritura de métodos

- Despacho dinámico (*dynamic dispatch*)

20.3.5. Ligadura dinámica

20.3.6. super

20.3.7. Sobreescritura de constructores

20.3.8. El problema del diamante con polimorfismo

20.3.9. Igualdad polimórfica

20.3.10. Sobreescritura de `__eq__`

20.4. Abstracción

20.4.1. Clases abstractas y métodos abstractos

20.5. Herencia vs. composición

21 INTERFACES GRÁFICAS DE USUARIO #ce5f#ce5g#ce5h#ev3#ra5 (est: 2027-03-30)

21.1. Introducción a Tkinter

21.1.1. ¿Qué es Tkinter?

21.1.2. Instalación y primeras pruebas

21.1.3. La ventana principal (Tk)

21.1.4. El bucle principal (*mainloop*)

21.2. Widgets básicos

21.2.1. Label

21.2.2. Button

21.2.3. Entry

21.2.4. Text

21.2.5. Checkbutton

21.2.6. Radiobutton

21.2.7. Atributos comunes

- Texto (text)
- Color (fg y bg)
- Fuente (font)
- Tamaño (width y height)

21.2.8. Métodos útiles

- get
- insert
- delete

21.3. *Layout* y organización de la interfaz

21.3.1. Gestión de geometría con pack, grid y place

21.3.2. Uso de Frame para dividir la ventana

21.3.3. Diseño *responsive* básico

21.4. Eventos y funciones asociadas

21.4.1. Asociar funciones a eventos (*callbacks*)

- Uso de command=
- Eventos con bind

21.4.2. Variables de control

22

BASES DE DATOS ORIENTADAS A OBJETOS #ce8a #ce8b #ce8c #ce8d #ce8e #ce8f #ce8g #ce8h
#ev3 #ra8 (est: 2027-04-06)

22.1. Introducción a las bases de datos orientadas a objetos (OODB)

22.1.1. Conceptos clave

- Objetos, clases y herencia
- Persistencia de objetos
- Integración de datos y comportamiento

22.1.2. Comparación con RDBMS

- Modelos de datos relacionales vs. orientados a objetos
- Ventajas y desventajas de OODB
- Escenarios donde es preferible usar una OODB

22.1.3. Casos de uso comunes

- Aplicaciones de simulación
- Modelos de datos complejos
- Aplicaciones con estructuras de datos jerárquicas

22.2. Persistencia de objetos en Python

22.2.1. Serialización y deserialización de objetos (módulo pickle)

22.2.2. JSON

22.3. Conceptos de bases de datos orientadas a objetos

22.3.1. Modelado de una OODB

- Clases como tablas
- Relaciones entre objetos (1 a 1, 1 a muchos, muchos a muchos)
- Polimorfismo en la base de datos

22.3.2. Implementación básica en Python

- Guardar objetos en una estructura persistente
- Simular una base de datos orientada a objetos sencilla

22.3.3. Limitaciones de una implementación casera

- Consistencia de datos
- Concurrencia
- Integridad referencial

22.4. Introducción a Zope Object Database (ZODB)

22.4.1. ¿Qué es ZODB?

- Ventajas de usar ZODB
- Comparación con otras soluciones de bases de datos

22.4.2. Instalación y configuración

- Estructura básica de un proyecto con ZODB

22.4.3. Persistencia de objetos en ZODB

- Crear una base de datos en ZODB
- Guardar, actualizar y eliminar objetos
- Administración de la base de datos y control de versiones

23 Bases de Datos Relacionales

(est: 2027-04-13)

23.1. Controlador JDBC

23.1.1. Instalación

23.1.2. CLASSPATH

- En consola
- En Visual Studio Code

23.2. Programa de ejemplo

23.3. Ejecutar programa

23.3.1. En consola

23.3.2. En Visual Studio Code

23.4. Establecimiento de conexiones

23.5. Recuperación de información

23.5.1. Ejecución de consultas

23.5.2. Selección de registros

23.5.3. Uso de parámetros

23.6. Manipulación de la información

23.6.1. Altas, bajas y modificaciones

4.3. Correspondencia con resultados de aprendizaje y criterios de evaluación

El símbolo «X» representa que en esa unidad didáctica se trabaja dicho resultado de aprendizaje pero no es el elemento fundamental de evaluación.

Unidades didácticas	#ra1	#ra2	#ra3	#ra4	#ra5	#ra6	#ra7	#ra8	#ra9
1. Introducción	#ce1a	#ce2a							
2. Lenguajes de programación	#ce1b #ce1c	#ce2a							
3. Expresiones	#ce1d #ce1e #ce1f #ce1g	#ce2b #ce2c #ce2d #ce2e #ce2f #ce2g #ce2h #ce2i							
4. Tipos y operaciones	#ce1d #ce1f #ce1g #ce1h								
5. Programación funcional	#ce1e #ce1f #ce1i								
6. Abstracciones funcionales				#ce4d					
7. El modelo de entorno	#ce1d #ce1e #ce1f #ce1g								
8. Programación imperativa (I)	#ce1e								
9. Programación imperativa (II)					#ce5a #ce5c				

Unidades didácticas	#ra1	#ra2	#ra3	#ra4	#ra5	#ra6	#ra7	#ra8	#ra9
10. Programación estructurada			#ce3a #ce3b #ce3c #ce3d #ce3e #ce3f #ce3g #ce3i						
11. Programación procedimental			#ce3a #ce3b #ce3c #ce3d #ce3e #ce3f #ce3g #ce3h #ce3i						
12. Procesamiento de datos de alto nivel						#ce6b #ce6c #ce6e #ce6f #ce6g			
13. Secuencias					#ce5b	#ce6a #ce6b #ce6c #ce6d #ce6h			
14. Colecciones no secuenciales						#ce6b #ce6c #ce6i #ce6j			
15. Entrada y salida por archivos					#ce5c #ce5d #ce5e				
16. Programación modular				#ce4i					
17. Abstracción de datos				#ce4b #ce4e					
18. Programación orientada a objetos (I)			#ce3h	#ce4a #ce4b #ce4c #ce4d #ce4e #ce4f #ce4g #ce4i		#ce6g			

Unidades didácticas	#ra1	#ra2	#ra3	#ra4	#ra5	#ra6	#ra7	#ra8	#ra9
19. Programación orientada a objetos (II)			#ce3h	#ce4a #ce4b #ce4c #ce4d #ce4e #ce4f #ce4g #ce4i		#ce6g			
20. Relaciones entre clases				#ce4a #ce4b #ce4c #ce4d #ce4e #ce4f #ce4g #ce4h #ce4i			#ce7a #ce7b #ce7c #ce7d #ce7e #ce7f #ce7g #ce7h #ce7i #ce7j		
21. Interfaces gráficas de usuario					#ce5f #ce5g #ce5h				
22. Bases de datos orientadas a objetos								#ce8a #ce8b #ce8c #ce8d #ce8e #ce8f #ce8g #ce8h	
23. Bases de datos relacionales									#ce9a #ce9b #ce9c #ce9d #ce9e #ce9f #ce9g

5. Evaluación y calificación

La evaluación tendrá como finalidad determinar el nivel de competencia de los alumnos y la consecución de los objetivos del módulo. Se desarrollará de forma objetiva, continua, formativa e

integradora de las competencias adquiridas en el centro y en la empresa u organismo equiparado, y atenderá a los siguientes aspectos:

- Aprendizaje autónomo, viendo la capacidad del alumno para interiorizar, gestionar y participar en los procesos de aprendizaje propios.
- Comprensión del lenguaje común.
- Adquisición de conceptos básicos del módulo profesional que permiten al alumno incluirlos como un elemento más de su realidad profesional.
- Participación y trabajo en grupo, viendo la capacidad que tiene este de escuchar y debatir las diferentes soluciones de un problema.
- Nivel de abstracción alcanzado.

Durante el desarrollo del curso existirán, al menos, una evaluación inicial, dos evaluaciones parciales y dos evaluaciones finales.

La evaluación se realizará atendiendo a los criterios de evaluación asociados a cada resultado de aprendizaje del módulo. Para ello, se usarán instrumentos de evaluación que medirán si el alumnado cumple con los criterios de evaluación y en qué grado.

- En cada **evaluación parcial** se medirá el grado de *cumplimiento* de los *criterios de evaluación* trabajados y evaluados en el trimestre correspondiente.

Se entenderá que el alumno **cumple** un determinado **criterio de evaluación** si supera al menos un instrumento de evaluación que mida el grado de cumplimiento de dicho criterio de evaluación.

- En cada **evaluación final** se determinará si el alumnado *alcanza*, y en qué medida, los *resultados de aprendizaje* del módulo a partir de los criterios de evaluación trabajados y evaluados que estén asociados a cada resultado de aprendizaje y en función de la ponderación que tenga cada criterio de evaluación en su correspondiente resultado de aprendizaje, según se establece en el apartado 5.3.

Para la superación del módulo se requerirá que el alumno **alcance** todos los **resultados de aprendizaje** del módulo.

Para que el seguimiento de dicha evaluación sea factible, el alumno deberá asistir con regularidad a las clases y participar activamente en las mismas, de forma que una sistemática y frecuente falta de asistencia a clase supondrá para el alumno la pérdida de la evaluación continua y sólo tendrá derecho a una prueba final. Asimismo, se requiere que el alumno **acceda al menos diariamente a las plataformas iPasen y Moodle Centros** y que **revise diariamente su correo en el dominio @iesdonana.org** para informarse puntualmente de las novedades que pudieran darse en el módulo, quedando claro que **es responsabilidad del alumno informarse activamente sobre las mismas**.

5.1. Valoración general de los instrumentos de evaluación y calificación

Los instrumentos de evaluación y calificación usados durante el curso para medir el grado de cumplimiento de los criterios de evaluación se clasifican en dos grandes tipos:

- Trabajos, actividades y ejercicios (casa, clase, grupo) (tipo **TRA**).
- Pruebas evaluativas (tipo **EXA**).

La evaluación de cualquiera de los trabajos, actividades, ejercicios y pruebas evaluativas podrá requerir, a criterio y discreción del profesor del módulo, la defensa de los mismos en una entrevista individual con el alumno para garantizar la autoría y la adquisición adecuada de las competencias correspondientes.

Los trabajos, actividades y/o ejercicios (**TRA**) versarán sobre los contenidos trabajados en una unidad didáctica o bloque de unidades didácticas conceptualmente relacionadas. (Esto significa, en consecuencia, que no es obligatoria la realización de trabajos, actividades y/o ejercicios en cada unidad didáctica, sino que a tales efectos se pueden agrupar varias unidades didácticas.)

Dentro de este grupo podrá incluirse alguna prueba (tipo test o de respuestas cortas) a responder de forma individual sobre conocimientos teóricos de determinados aspectos básicos asociados a unidades (o conjunto de unidades) didácticas concretas.

Asimismo, se realizará un examen (**EXA**) al final de las evaluaciones parciales primera y segunda, coincidiendo aproximadamente con el final del trimestre correspondiente. Debido al carácter de evaluación continua del módulo, así como del hecho de que cada contenido trabajado se asienta sobre los anteriores, es posible que el alumno requiera, para la evaluación positiva de un trimestre, el conocimiento necesario de contenidos de trimestres anteriores.

Cada examen, a su vez, constará de dos partes bien diferenciadas:

- **Parte teórica:** Una prueba que versará sobre conocimientos esenciales con preguntas tipo test, de respuestas cortas o similar (**TEO**).
- **Parte práctica:** Una prueba práctica con problemas y ejercicios donde el alumno deberá aplicar lo aprendido escribiendo código real (**PRA**).

La calificación del examen (**EXA**), según se recoge en el algoritmo 1, se valorará de 0,0 a 10,0 y se calculará como una media ponderada de ambas partes **TEO** y **PRA**, de forma que para superar el examen será necesario obtener una calificación mínima de 3,0 en la parte teórica y de 4,0 en la parte práctica; en caso contrario, la calificación del examen no podrá ser superior a 4,0.

Algoritmo 1 (Cálculo de la calificación del examen)

$$M = 0,3 \cdot \text{TEO} + 0,7 \cdot \text{PRA}$$

$$\text{EXA} = \begin{cases} M & \text{si } \text{TEO} \geq 3,0 \text{ y } \text{PRA} \geq 4,0 \\ \min(M; 4,0) & \text{e. o. c.} \end{cases}$$

Donde:

M = Media ponderada entre teoría y práctica.

TEO = Calificación de la parte teórica del examen, valorada del 0,0 al 10,0.

PRA = Calificación de la parte práctica del examen, valorada del 0,0 al 10,0.

EXA = Calificación del examen, valorada del 0,0 al 10,0, calculada como la media ponderada de **TEO** y **PRA**.

5.2. Calificación

5.2.1. De un criterio de evaluación

La calificación de cada criterio de evaluación, una vez trabajado y evaluado, se valorará de 0,0 a 10,0 y se calculará como la calificación obtenida en el instrumento de evaluación y calificación usado para medir el grado de cumplimiento de ese criterio de evaluación.

Se entenderá que el alumno **cumple un criterio de evaluación** si la calificación que ha obtenido en el mismo es igual o superior a 4,5.

Si, a lo largo del curso, se usaran varios instrumentos para evaluar y calificar el mismo criterio de evaluación, la calificación de dicho criterio de evaluación se calcularía de la siguiente forma:

1. Se agruparán las calificaciones obtenidas en los diferentes instrumentos que hayan medido ese criterio de evaluación en función del tipo de instrumento usado para evaluarlo (**TRA** o **EXA**). Se obtendrán así dos grupos de calificaciones, uno por cada tipo de instrumento.
2. En cada grupo, se seleccionará la calificación máxima dentro de ese grupo.

3. Se calculará la media ponderada (M) de las dos calificaciones resultantes del punto anterior, atendiendo a las siguientes ponderaciones:

TRA	50 %
EXA	50 %

Si, por algún motivo, no se usaran instrumentos de uno de los dos tipos (**TRA** o **EXA**) durante la evaluación, el otro apartado soportaría el 100 % de la carga evaluativa, de forma que la evaluación y calificación resultaría únicamente de dicho apartado.

4. Se calcula la calificación del criterio de evaluación de la siguiente forma, que garantiza que un criterio de evaluación califique como cumplido cuando haya al menos un instrumento con una calificación mayor o igual a 4,5 que mida ese criterio de evaluación:

$$CE_{i,j} = \begin{cases} \max(M_{ij}; 4,5) & \text{si } \max I_{ij} \geq 4,5 \\ M_{ij} & \text{e. o. c.} \end{cases}$$

Donde:

$CE_{i,j}$ = Calificación del criterio de evaluación j asociado al resultado de aprendizaje i .

M_{ij} = Media ponderada calculada en el punto 3 anterior para el criterio de evaluación j asociado al resultado de aprendizaje i .

I_{ij} = Conjunto de todas las calificaciones obtenidas por el alumno en todos los instrumentos de evaluación y calificación que miden el criterio de evaluación j asociado al resultado de aprendizaje i .

En consecuencia:

- La calificación de un determinado criterio de evaluación podría ir cambiando a lo largo del curso en función de los instrumentos de evaluación y calificación realizados por el alumno para evaluar, recuperar, mejorar su calificación, etcétera.
- La calificación de un criterio de evaluación no podrá ser inferior a 4,5 si el alumno ha obtenido una calificación igual o superior a 4,5 en algún instrumento de evaluación y calificación que haya medido ese criterio de evaluación.

5.2.2. De un resultado de aprendizaje

La calificación de cada resultado de aprendizaje, según se recoge en el algoritmo 2, se valorará de 0,0 a 10,0 y se calculará como una media ponderada de los criterios de evaluación trabajados y evaluados que estén asociados a ese resultado de aprendizaje, siendo requisito imprescindible

para alcanzar un resultado de aprendizaje el cumplir todos los criterios de evaluación trabajados y evaluados; en caso contrario, la calificación del resultado de aprendizaje no podrá ser superior a 4,0.

Algoritmo 2 (Cálculo de la calificación de un resultado de aprendizaje)

$$M_i = \frac{\sum_{j \in C_i} \mathbf{CE}_{i \cdot j} \cdot p_{ij}}{\sum_{j \in C_i} p_{ij}}$$

$$\mathbf{RA}_i = \begin{cases} M_i & \text{si } \min_{c \in C_i} \{c\} \geq 4,5 \\ \min(M_i; 4,0) & \text{e. o. c.} \end{cases}$$

Donde:

M_i = Media ponderada de los criterios de evaluación asociados al resultado de aprendizaje i .

$\mathbf{RA}_i$ = Calificación del resultado de aprendizaje i .

$\mathbf{CE}_{i \cdot j}$ = Calificación del criterio de evaluación j asociado al resultado de aprendizaje i .

C_i = Conjunto de las calificaciones de los criterios de evaluación trabajados y evaluados, asociados al resultado de aprendizaje i .

p_{ij} = Ponderación del criterio de evaluación j en el resultado de aprendizaje i , según se determina en el apartado 5.3.

Se entenderá que el alumno ha **alcanzado un resultado de aprendizaje** si la calificación que ha obtenido en el mismo es igual o superior a 4,5.

5.2.3. Calificaciones parciales

La calificación del módulo en cada evaluación parcial (primer y segundo trimestres por separado), según se recoge en el algoritmo 3, se valorará de 0,0 a 10,0 y se calculará como la media aritmética de las calificaciones de los criterios de evaluación trabajados y evaluados durante el trimestre correspondiente, siendo requisito imprescindible para aprobar una evaluación parcial el cumplir todos los criterios de evaluación trabajados y evaluados; en caso contrario, la calificación de la evaluación parcial no podrá ser superior a 4,0.

Algoritmo 3 (Cálculo de una calificación parcial)

$$M_i = \frac{\sum_{c \in C_i} c}{|C_i|}$$

$$P_i = \begin{cases} M_i & \text{si } \min_{c \in C_i} \{c\} \geq 4,5 \\ \min(M_i; 4,0) & \text{e. o. c.} \end{cases}$$

Donde:

M_i = Media aritmética de las calificaciones de los criterios de evaluación trabajados en el trimestre $i \in \{1,2\}$.

C_i = Conjunto de las calificaciones de los criterios de evaluación trabajados y evaluados en el trimestre $i \in \{1,2\}$.

P_i = Calificación parcial del trimestre $i \in \{1,2\}$.

Se entenderá que el alumno ha **aprobado** una **evaluación parcial** si la calificación que ha obtenido en la misma es igual o superior a 4,5. La calificación introducida en el sistema Séneca será la calificación parcial redondeada al entero más cercano, y no podrá ser inferior a 1,0.

Los criterios de evaluación relativos al tercer trimestre serán evaluados en la primera sesión de evaluación final.

5.2.4. Calificación final

La calificación del módulo en cualquiera de las dos evaluaciones finales se calculará como la media aritmética de las calificaciones de los diferentes resultados de aprendizaje, según se recoge en el algoritmo 4.

Algoritmo 4 (Cálculo de la calificación final)

$$M = \frac{\sum_{r \in R} r}{|R|}$$

$$F = \begin{cases} M & \text{si } \min_{r \in R} \{r\} \geq 4,5 \\ \min(M; 4,0) & \text{e. o. c.} \end{cases}$$

Donde:

M = Media aritmética de las calificaciones de los resultados de aprendizaje del módulo.

R = Conjunto de las calificaciones de los resultados de aprendizaje del módulo.

F = Calificación final del módulo.

La calificación introducida en el sistema Séneca será la calificación final redondeada al entero más cercano, y no podrá ser inferior a 1,0. Asimismo, si el alumno no ha alcanzado todos los resultados de aprendizaje, la calificación final del módulo no podrá ser superior a 4,0.

5.3. Ponderación de los criterios de evaluación

Los criterios de evaluación de cada resultado de aprendizaje se ponderan con los siguientes pesos relativos:

Resultado de aprendizaje (RA_i)	Criterio de evaluación ($CE_{i,j}$)	Porcentaje (p_{ij})
RA_1 #ra1	$CE_{1,a}$ #ce1a	11,1 %
	$CE_{1,b}$ #ce1b	11,1 %
	$CE_{1,c}$ #ce1c	11,1 %
	$CE_{1,d}$ #ce1d	11,1 %
	$CE_{1,e}$ #ce1e	11,1 %
	$CE_{1,f}$ #ce1f	11,1 %
	$CE_{1,g}$ #ce1g	11,1 %
	$CE_{1,h}$ #ce1h	11,1 %
	$CE_{1,i}$ #ce1i	11,1 %
RA_2 #ra2	$CE_{2,a}$ #ce2a	11,1 %
	$CE_{2,b}$ #ce2b	11,1 %
	$CE_{2,c}$ #ce2c	11,1 %

Resultado de aprendizaje (RA _i)	Criterio de evaluación (CE _{i·j})	Porcentaje (p _{ij})
RA ₂ #ra2	CE _{2·d} #ce2d	11,1 %
	CE _{2·e} #ce2e	11,1 %
	CE _{2·f} #ce2f	11,1 %
	CE _{2·g} #ce2g	11,1 %
	CE _{2·h} #ce2h	11,1 %
	CE _{2·i} #ce2i	11,1 %
RA ₃ #ra3	CE _{3·a} #ce3a	11,1 %
	CE _{3·b} #ce3b	11,1 %
	CE _{3·c} #ce3c	11,1 %
	CE _{3·d} #ce3d	11,1 %
	CE _{3·e} #ce3e	11,1 %
	CE _{3·f} #ce3f	11,1 %
	CE _{3·g} #ce3g	11,1 %
	CE _{3·h} #ce3h	11,1 %
	CE _{3·i} #ce3i	11,1 %
RA ₄ #ra4	CE _{4·a} #ce4a	11,1 %
	CE _{4·b} #ce4b	11,1 %
	CE _{4·c} #ce4c	11,1 %
	CE _{4·d} #ce4d	11,1 %
	CE _{4·e} #ce4e	11,1 %
	CE _{4·f} #ce4f	11,1 %
	CE _{4·g} #ce4g	11,1 %
	CE _{4·h} #ce4h	11,1 %
	CE _{4·i} #ce4i	11,1 %
RA ₅ #ra5	CE _{5·a} #ce5a	12,5 %
	CE _{5·b} #ce5b	12,5 %
	CE _{5·c} #ce5c	12,5 %
	CE _{5·d} #ce5d	12,5 %
	CE _{5·e} #ce5e	12,5 %
	CE _{5·f} #ce5f	12,5 %
	CE _{5·g} #ce5g	12,5 %
	CE _{5·h} #ce5h	12,5 %
RA ₆ #ra6	CE _{6·a} #ce6a	10,0 %
	CE _{6·b} #ce6b	10,0 %
	CE _{6·c} #ce6c	10,0 %
	CE _{6·d} #ce6d	10,0 %
	CE _{6·e} #ce6e	10,0 %
	CE _{6·f} #ce6f	10,0 %
	CE _{6·g} #ce6g	10,0 %
	CE _{6·h} #ce6h	10,0 %
	CE _{6·i} #ce6i	10,0 %

Resultado de aprendizaje (RA _i)	Criterio de evaluación (CE _{i·j})	Porcentaje (p _{ij})
RA ₆ #ra6	CE _{6·j} #ce6j	10,0 %
RA ₇ #ra7	CE _{7·a} #ce7a	10,0 %
RA ₇ #ra7	CE _{7·b} #ce7b	10,0 %
RA ₇ #ra7	CE _{7·c} #ce7c	10,0 %
RA ₇ #ra7	CE _{7·d} #ce7d	10,0 %
RA ₇ #ra7	CE _{7·e} #ce7e	10,0 %
RA ₇ #ra7	CE _{7·f} #ce7f	10,0 %
RA ₇ #ra7	CE _{7·g} #ce7g	10,0 %
RA ₇ #ra7	CE _{7·h} #ce7h	10,0 %
RA ₇ #ra7	CE _{7·i} #ce7i	10,0 %
RA ₇ #ra7	CE _{7·j} #ce7j	10,0 %
RA ₈ #ra8	CE _{8·a} #ce8a	12,5 %
RA ₈ #ra8	CE _{8·b} #ce8b	12,5 %
RA ₈ #ra8	CE _{8·c} #ce8c	12,5 %
RA ₈ #ra8	CE _{8·d} #ce8d	12,5 %
RA ₈ #ra8	CE _{8·e} #ce8e	12,5 %
RA ₈ #ra8	CE _{8·f} #ce8f	12,5 %
RA ₈ #ra8	CE _{8·g} #ce8g	12,5 %
RA ₈ #ra8	CE _{8·h} #ce8h	12,5 %
RA ₉ #ra9	CE _{9·a} #ce9a	14,2 %
RA ₉ #ra9	CE _{9·b} #ce9b	14,2 %
RA ₉ #ra9	CE _{9·c} #ce9c	14,2 %
RA ₉ #ra9	CE _{9·d} #ce9d	14,2 %
RA ₉ #ra9	CE _{9·e} #ce9e	14,2 %
RA ₉ #ra9	CE _{9·f} #ce9f	14,2 %
RA ₉ #ra9	CE _{9·g} #ce9g	14,2 %

5.4. Medidas de recuperación y mejora de la calificación final

A lo largo del curso (idealmente, a comienzos del segundo y tercer trimestres) se establecerán instrumentos de recuperación destinados a que el alumnado cumpla los criterios de evaluación trabajados y evaluados en trimestres anteriores que no hayan podido cumplir hasta ahora al no haber podido superar los instrumentos de evaluación correspondientes.

Los criterios de evaluación que se trabajen en unidades didácticas temporalizadas en varios trimestres podrían no necesitar de instrumentos de recuperación, ya que el alumno tendría varias oportunidades para cumplir el mismo criterio de evaluación a lo largo de varios trimestres con los instrumentos de evaluación propios de las unidades didácticas programadas para esos trimestres.

Los instrumentos de recuperación utilizados para que el alumnado pueda cumplir los criterios de evaluación que aún no ha cumplido, podrán coincidir o no con los utilizados anteriormente para evaluar el cumplimiento de tales criterios, a discreción del profesor. En particular, se podrán usar pruebas evaluativas (instrumentos del tipo **EXA**) como instrumentos de recuperación para que el alumnado pueda cumplir criterios de evaluación que aún no haya cumplido y que previamente se habían medido con instrumentos del tipo **TRA**.

5.4.1. Prueba final

Se establece la existencia de un examen o prueba en cada evaluación final para aquellos alumnos que no hayan podido superar el módulo mediante el procedimiento habitual de evaluación continua y que, por tanto, tengan pendiente alcanzar algún resultado de aprendizaje que no se haya podido alcanzar ni siquiera mediante los instrumentos de recuperación indicados anteriormente.

Esta prueba final permite al alumnado la superación del módulo incluso en el caso de no haber asistido con regularidad a clase a lo largo del curso. Es, por tanto, el único instrumento que posibilita el que un alumno que cuente con un número de faltas de asistencia superior al permitido (y que, por consiguiente, haya perdido el derecho a la evaluación continua) pueda demostrar la consecución de los objetivos del módulo y, en consecuencia, su superación.

Dicha prueba o examen final constará de varias partes separadas. De ellas, las partes que el alumno deberá realizar y superar dependerán de las calificaciones que haya obtenido en los diferentes resultados de aprendizaje a lo largo del curso, de manera que el alumno deberá realizar las partes correspondientes a los resultados de aprendizaje que aún no haya alcanzado.

Finalmente, la calificación obtenida en la prueba final se tendrá en cuenta para calcular la **calificación final del módulo** según se establece en 5.2.4.

El alumno que, debiendo presentarse a la prueba final, no lo haga, será calificado como «No evaluado» (NE).

Existirá una prueba final por cada una de las dos evaluaciones finales del curso, de forma que el alumno deberá presentarse, al menos, a la primera de ellas para evaluar, al menos, los criterios de evaluación relativos al tercer trimestre, así como todos los demás criterios de evaluación que no hubiera cumplido a lo largo del curso.

5.4.2. Mejora de la calificación final

El alumno que, habiendo superado el módulo mediante el procedimiento habitual de evaluación continua, desee mejorar su calificación final («subir nota»), podrá hacerlo en la primera

prueba final, manteniéndose la calificación final más alta de entre las obtenidas antes y después de presentarse a subir nota en dicha prueba final. A tal efecto, se indicará en la prueba final qué partes deberá realizar el alumnado que se haya presentado a subir nota.

6. Evaluación de la práctica docente

Se realizarán entre el alumnado tres encuestas de valoración trimestrales y anónimas, al final de cada trimestre, formadas por preguntas cuantitativas y cualitativas relacionadas con aspectos que afecten al módulo, su organización, su evaluación, los materiales empleados, el día a día de su práctica docente y el profesorado encargado de su docencia. A lo largo del curso se analizará la evolución de los resultados obtenidos globalmente y por cada pregunta para observar su progresión positiva o negativa y así poder tomar las acciones que se estime oportunas llegado el caso.

7. Orientaciones pedagógicas

Este módulo profesional contiene parte de la formación necesaria para desempeñar la función de **programación de aplicaciones de propósito general en lenguajes orientados a objetos**.

La **función** de programación de aplicaciones de propósito general en lenguajes orientados a objetos incluye aspectos como:

- El desarrollo de programas organizados en clases aplicando los principios de la programación orientada a objetos.
- La utilización de interfaces para la interacción de la aplicación con el usuario.
- La identificación, análisis e integración de librerías para incorporar funcionalidades específicas a los programas desarrollados.
- El almacenamiento y recuperación de información en sistemas gestores de bases de datos relacionales y orientados a objetos.

Las **actividades profesionales** asociadas a esta función se aplican en el desarrollo y la adaptación de programas informáticos de propósito general en lenguajes orientados a objetos.

Las **líneas de actuación** en el proceso de enseñanza-aprendizaje que permiten alcanzar los objetivos del módulo profesional versarán sobre:

- La interpretación y aplicación de los principios de la programación orientada a objetos.

- La evaluación, selección y utilización de herramientas y lenguajes de programación orientados a objetos.
- La utilización de las características específicas de lenguajes y entornos de programación en el desarrollo de aplicaciones informáticas.
- La identificación de las funcionalidades aportadas por los sistemas gestores de bases de datos y su incorporación a los programas desarrollados.
- La prueba, depuración y documentación de los programas desarrollados.

8. Orientaciones metodológicas

El módulo es totalmente práctico, y el proceso de enseñanza-aprendizaje se fundamenta en la interacción continua y total de los alumnos con las herramientas software utilizadas y estudiadas a lo largo del curso.

El módulo construye el aprendizaje de forma progresiva, comenzando con el estudio de los fundamentos de la programación en un núcleo funcional sencillo para, desde ahí, ir incorporando nuevos elementos paulatinamente aumentando poco a poco la complejidad, añadiendo primero los elementos básicos de la programación imperativa, introduciendo luego las ventajas de la programación estructurada y, finalmente, incorporando los mecanismos de la programación orientada a objetos. La idea, por tanto, es hacer que cada nueva capa de complejidad se apoye en la anterior.

De la misma forma, el estudio de los datos se hará añadiendo complejidad progresiva, partiendo de los tipos de datos más básicos, introduciendo luego los tipos compuestos, estudiando los tipos abstractos de datos y acabando con la implementación de las estructuras de datos clásicas. Es en ese punto donde este enfoque se encuentra con el del párrafo anterior, al hacer notar que las clases del paradigma de programación orientada a objetos son, en esencia, implementaciones de tipos abstractos de datos.

Al disponer de ocho horas a la semana, repartidas entre cuatro días a dos horas por día, el objetivo en todo momento será usar los dos primeros días para exponer contenidos y resolver dudas, y los otros dos días para realizar ejemplos y ejercicios en clase. Para ello, será necesario que el alumno previamente haya trabajado en casa los contenidos que se vayan a impartir en esa semana, usando los materiales y apuntes que el profesor pone previamente a su disposición.

Para servir de vehículo de comunicación y soporte de los conceptos aprendidos durante el curso se utilizará el lenguaje de programación Python. Los motivos que justifican el uso de Python como lenguaje de introducción a la programación son los siguientes:

- Es un lenguaje sencillo y fácil de aprender en cuanto a sus aspectos básicos.

- Es el lenguaje más usado por universidades y centros de formación en cursos de introducción a la programación.
- Es un lenguaje interpretado y dispone de intérprete interactivo, lo que facilita el desarrollo iterativo e incremental.
- Es un lenguaje de tipado dinámico pero dispone de mecanismos opcionales de tipado estático, por lo que pasar de uno a otro resulta sencillo.
- Es un lenguaje multiparadigma, lo que permite transitar por los estilos funcional, imperativo, estructurado y orientado a objetos.
- Es, a día de hoy, el lenguaje con mayor crecimiento en la industria del software, superando a Java y JavaScript.
- Es muy usado en la industria y dispone de muchas y probadas herramientas de desarrollo, así como librerías y paquetes de muy variada funcionalidad.
- Permite cubrir perfectamente todos los resultados de aprendizaje y criterios de evaluación del módulo profesional.

De manera transversal, se hará hincapié continuamente en el aseguramiento de la calidad del producto resultante así como del proceso de desarrollo y el código fuente desarrollado, poniendo especial énfasis en la metodología *TDD (Test-Driven Development)* y las pruebas automáticas.

La enseñanza se basará casi por completo en el estudio previo de los conceptos básicos y suficientes para la realización de ejercicios y supuestos de aplicación, que obliguen al alumno a enfrentarse con las herramientas software necesarias para solucionarlos.

Las actividades propuestas podrán ser individuales o grupales, aplicando donde corresponda el concepto de *programación en pareja*¹ como vehículo de colaboración y aprendizaje compartido entre varios alumnos.

Finalmente, se incentivará al alumno para que mejore su comprensión mediante el autoaprendizaje y la elaboración propia de ejercicios y desarrollos.

9. Recursos

9.1. Hardware

- Un ordenador para cada alumno, conectado a la red local del aula y esta, a su vez, a la troncal del Centro.
- Conexión a Internet de banda ancha.
- Cañón retroproyector.

¹ https://es.wikipedia.org/wiki/Programaci%C3%B3n_en_pareja

9.2. Software

- Sistema operativo GNU/Linux.
- El resto de herramientas y aplicaciones necesarias se instalarán a través de Internet a lo largo del curso.

9.3. Online

- Plataforma **Moodle Centros**² de la Junta de Andalucía para el seguimiento general del módulo, incluyendo distribución de material y entrega de ejercicios y exámenes.
- **GitHub**³ como herramienta centralizada para compartir código y para la gestión integral de todo elemento satélite del mismo (control de versiones, desarrollo colaborativo, incidencias, etcétera).

9.4. Bibliografía

9.4.1. Principal

- Apuntes y documentación *online*⁴ suministrados por el profesor.
- Documentación oficial de Python⁵.

9.4.2. Complementaria

- ABELSON, H., SUSSMAN, G. J. Y SUSSMAN, J. (1996). *Structure and Interpretation of Computer Programs (2nd edition)*. Cambridge: MIT Press.
- PEÑA, R. (2003). *Diseño de programas: formalismo y abstracción (2.ª edición)*. Madrid: Prentice-Hall.
- BLANCO, J., SMITH, S., BARSOTTI, D. (2009). *Cálculo de Programas*. Córdoba (Argentina): Fa.M.A.F., Universidad Nacional de Córdoba.
- PAREJA, C., OJEDA, M., ANDEYRO, Á. L., ROSSI, C. (1997). *Desarrollo de algoritmos y técnicas de Programación en Pascal*. Madrid: Ra-Ma.
- JOYANES AGUILAR, L. (2008). *Fundamentos de programación. Algoritmos, estructuras de datos y objetos*. Aravaca: McGraw-Hill Interamericana de España.

² <https://educacionadistancia.juntadeandalucia.es/centros/cadiz/>

³ <https://github.com>

⁴ <https://pro.iesdonana.org>

⁵ <https://docs.python.org/3/>

- VAN-ROY, P., HARIDI, S. (2004). *Concepts, techniques, and models of computer programming*. Cambridge, Mass: MIT Press.

10. Atención a la diversidad

Se llevarán a cabo actividades de refuerzo o ampliación para aquellos alumnos que así lo requieran en función de las necesidades detectadas:

- Para los alumnos que muestren dificultades de aprendizaje, se propondrán **actividades de refuerzo** destinadas a afianzar los aspectos conceptuales y procedimentales en los que el alumnado presente carencias.
- Para los alumnos que muestren un mayor grado de adquisición de competencias, se propondrán **actividades de ampliación** que supongan la investigación autónoma de contenidos opcionales (aquellos marcados con la etiqueta *#opcional*).

11. Temas transversales

Con el objeto de fomentar entre los alumnos el hábito de la lectura, se plantearán actividades individuales y en grupo en las que, para su resolución, se necesite leer información de distintas fuentes escritas, como artículos, blogs, páginas web, tutoriales, etc.

La evolución experimentada por la informática en los últimos años tiene como consecuencia su influencia inevitable en todos los aspectos de las relaciones entre las personas y entre éstas y el entorno. Además ha demostrado ser un medio valiosísimo para la educación cualquiera que sea el ámbito en el que se use. En concreto, en cuanto a los temas transversales propuestos:

- **Educación ambiental:** La utilización de la informática, en general, y sobre todo en los negocios, hace que grandes volúmenes de información puedan ser almacenados en soportes informáticos, discos, CD, ... y enviados de unos lugares a otros a través de las redes informáticas, evitándose de esta manera el consumo de grandes cantidades de papel y, por consiguiente, la destrucción de bosques, contribuyendo de alguna manera a la preservación de los medios naturales y medioambientales.
- **Educación del consumidor:** El análisis y la utilización de diferentes herramientas informáticas favorecen la capacidad del alumnado para decidir sobre los productos informáticos que debe adquirir y utilizar de manera ventajosa.
- **Educación para la salud:** Cuando se utilizan equipos informáticos se procura que el alumnado conozcan una serie de normas de higiene y seguridad en el trabajo, así como

sobre las precauciones necesarias en el empleo de los equipos. De esta manera, se intenta que el alumnado conozca los principios de la ergonomía del puesto de trabajo, para que cualquier trabajo frente al ordenador resulte lo más agradable posible y no le cause ningún problema. En este sentido, resultan de interés las instrucciones elaboradas por el Instituto Nacional de Seguridad e Higiene en el Trabajo [12].

- **Educación para la igualdad de oportunidades entre ambos sexos:** Desde este módulo contamos con elementos para concienciar al alumnado sobre la igualdad de oportunidades para alumnos y alumnas:
 - Formando grupos mixtos de trabajo.
 - Distribuyendo las tareas a realizar en la misma medida entre el alumnado de ambos sexos.
 - Haciendo que todos utilicen los mismos o equivalentes equipos.
 - Fomentando la participación de todos, sin distinciones de sexo.
- **Educación para el trabajo:** Respecto a este módulo encontramos los siguientes elementos:
 - Técnicas de trabajo en grupo: sujeción a unas reglas corporativas.
 - Colaboración de varias personas para la realización de un único trabajo.
- **Educación para la paz y la convivencia:** Se trabajan los elementos siguientes:
 - Acuerdos para la utilización de los mismos estándares en toda la comunidad internacional.
 - Respeto por las opiniones de los demás.
 - Aprender a escuchar.

12. Actuaciones para desarrollar la perspectiva de género

El conocimiento de la realidad existente es el primer paso a realizar para incorporar la perspectiva de género. De esta manera, se descubrirá la existencia de situaciones de desequilibrio entre mujeres y hombres en el desempeño de la actividad docente.

La perspectiva de género es trabajada de manera transversal y permanente en todas las Unidades Didácticas que componen esta programación. El IES Doñana como organización social en aplicación de esta óptica favorece, entre otros aspectos, la detección de estereotipos y la asignación de roles y responsabilidades, la evaluación del uso y control de los recursos puestos a

disposición de hombres y mujeres con la finalidad última de introducir las modificaciones y medidas correctoras necesarias para eliminar las desigualdades detectadas en cualquier ámbito de la vida del centro y particularmente dentro del aula.

En el marco de esta programación, el análisis de estas circunstancias permite identificar las diferentes necesidades, intereses y perspectivas de mujeres y hombres sobre las que diseñar estrategias que equiparen las oportunidades de ambas partes en las distintas actuaciones que lo integran. Fundamentalmente en los siguientes círculos se realizan las actuaciones:

- Profesores–profesores
- Alumnos–alumnos y
- Alumnos–profesores

Implica tener en cuenta las siguientes cuestiones:

1. Valorar la situación de partida de hombres y mujeres.
2. Analizar las necesidades y obligaciones relacionadas con la actividad cotidiana en el centro y la posición social de hombres y mujeres en el centro.
3. Velar por el cumplimiento de la condición de igualdad de género en todos los ámbitos de actuación como cuestión de justicia y responsabilidad social.

12.1. Actuaciones generales permanentes

1. Revisión del material curricular para la eliminación de la transmisión de estereotipos o modelos de conductas determinados por el género, tipo identificación cultural de funciones realizadas tradicionalmente por hombres o mujeres.
2. Detectar las desigualdades y discriminaciones de género existentes en el centro para su tratamiento/denuncia pertinente.
3. Garantizar la participación equilibrada de hombres y mujeres en las distintas actividades en el aula y en el centro.
4. Velar porque el contenido gráfico y lingüístico de las acciones, materiales y dispositivos de formación y difusión carezca de cualquier carácter o pretensión discriminatoria.
5. Participación en las actividades propuestas por el Plan de Igualdad del centro articulado a través de actuaciones propias o la acción tutorial:
 - a) 25 de noviembre: Día Internacional de la Eliminación de la Violencia contra la Mujer.
 - b) 30 de enero: Resolución de conflictos de forma pacífica. Día de la Paz.
 - c) 8 de marzo: Día Internacional de la Mujer.

Desde el primer momento se advertirá al alumnado que el uso del vocabulario y expresiones propias del lenguaje hablado y escrito se llevará a cabo de forma extensiva a ambos géneros, de manera que cuando hablamos del «administrador» o el «programador» lo hacemos siempre considerando que dichos roles son de aplicación a hombres y mujeres por igual. Así pues, resultará innecesario y, por tanto, se evitará el uso de fórmulas tales como «administrador o administradora», que recargan el lenguaje sin aportar información adicional. Ello además va en consonancia con lo manifestado por la Real Academia Española, al afirmar que:

*«El español dispone de un mecanismo inclusivo: el masculino gramatical, que, como término no marcado de la oposición de género, puede referirse a grupos formados de hombres y mujeres y, en contextos genéricos o inespecíficos, a personas de uno u otro sexo».*⁶

Por otra parte, en el planteamiento y realización de tareas y ejercicios, se procurará el equilibrio en cuanto a presencia de actores de ambos géneros.

13. Funciones del profesor de desdoble

Las funciones asignadas al profesor de desdoble serán las siguientes:

- Solventar dudas del alumnado sobre las actividades que se desarrollarán durante el curso.
- Impartir clases de refuerzo a los alumnos que se incorporan más tarde o necesitan un mayor esfuerzo para asimilar los conocimientos impartidos.
- Mantenimiento de los equipos informáticos o cualquier problema técnico que pueda surgir durante las clases.
- Si el profesor titular faltase, tomará el control de las clases de la tal forma que continúen con la mayor normalidad posible realizando las tareas que correspondan, bien sea impartición de clases, corrección de actividades o realización de exámenes.

El cuaderno del profesor estará compartido para que ambos profesores puedan acceder a él en caso necesario.

Referencias

- [1] Orden de 26 de septiembre de 2025, por la que se regula la fase de formación en empresa u organismo equiparado de los grados D y E del Sistema de Formación Profesional de la Comunidad Autónoma de Andalucía (BOJA n.º 187 C1 de 29 de septiembre).

⁶ <https://twitter.com/RAEinforma/status/1111565711653113856>

- [2] Orden de 18 de septiembre de 2025, por el que se regula la evaluación, certificación, acreditación y titulación académica del alumnado que cursa enseñanzas de los grados D y E del sistema de Formación Profesional en la Comunidad Autónoma de Andalucía (BOJA n.º 180 C1 de 18 de septiembre).
- [3] Decreto 147/2025, de 17 de septiembre de 2025, por el que se establece la ordenación de las enseñanzas de los Grados D y E del Sistema de Formación Profesional en la Comunidad Autónoma de Andalucía (BOJA n.º 179 C1 de 17 de septiembre).
- [4] Real Decreto 659/2023, de 18 de julio (BOE n.º 174 de 22 de julio), por el que se desarrolla la ordenación del Sistema de Formación Profesional.
- [5] Ley Orgánica 3/2022, de 31 de marzo (BOE n.º 78 de 1 de abril), de ordenación e integración de la Formación Profesional.
- [6] Real Decreto 1147/2011, de 29 de julio (BOE n.º 182 de 30 de julio) por el que se establece la ordenación general de la formación profesional del sistema educativo.
- [7] Real Decreto 500/2024, de 21 de mayo (BOE n.º 129 de 28 de mayo), por el que se modifican determinados reales decretos por los que se establecen títulos de Formación Profesional de grado superior y se fijan sus enseñanzas mínimas.
- [8] Real Decreto 405/2023, de 29 de mayo (BOE n.º 132 de 3 de junio), por el que se actualizan los títulos de la formación profesional del sistema educativo de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma y Técnico Superior en Desarrollo de Aplicaciones Web, de la familia profesional Informática y Comunicaciones, y se fijan sus enseñanzas mínimas.
- [9] Orden de 16 de junio de 2011, por la que se desarrolla el currículo correspondiente al título de Técnico Superior en Desarrollo de Aplicaciones Web (BOJA n.º 149 del 1 de agosto).
- [10] Orden EDU/2887/2010, de 2 de noviembre (BOE n.º 273 de 11 de noviembre), por la que se establece el currículo del ciclo formativo de Grado Superior correspondiente al título de Técnico Superior en Desarrollo de Aplicaciones Web.
- [11] Real Decreto 686/2010, de 20 de mayo (BOE n.º 143 de 12 de junio), por el que se establece el título de Técnico Superior en Desarrollo de Aplicaciones Web.
- [12] Instituto Nacional de Seguridad e Higiene en el Trabajo. *Instrucción básica para el trabajador usuario de pantallas de visualización de datos.*⁷

⁷ http://www.insht.es/InshtWeb/Contenidos/Documentacion/TextosOnline/Guias_Ev_Riesgos/Instruccion_Pantallas/Instruccion_basica.pdf